

Popular IoT operating systems

Several popular IoT operating systems have emerged to address the diverse needs of the IoT market.

TinyOS

TinyOS is a lightweight and efficient operating system specifically designed for resource-constrained devices. It is ideal for sensor networks and wireless communication applications, prioritising energy efficiency and low-latency communication. TinyOS is known for its small footprint and ability to operate on devices with minimal hardware resources.

Contiki

Contiki is an open source operating system with a strong focus on networking capabilities, particularly IPv6 support. It is well-suited for IoT devices that require robust networking features, such as those deployed in large-scale IoT networks. Contiki supports a variety of wireless protocols and network topologies, making it a versatile choice for diverse IoT applications.

FreeRTOS

FreeRTOS is a widely used real-time operating system known for its performance and reliability. It offers a small memory footprint and deterministic behaviour, making it suitable for applications that require precise timing and resource optimisation. FreeRTOS is commonly used in embedded systems, including IoT devices, due to its efficiency and ease of use.

Zephyr

Zephyr is an open source, scalable operating system that supports a wide range of hardware platforms. It provides a modular architecture and a rich set of device drivers, making it easy to customise and adapt to different IoT applications. Zephyr emphasises

security and reliability, making it suitable for critical IoT applications that require robust security measures.

RIOT OS

RIOT OS is a multi-threading operating system designed for resource-constrained devices. It supports various hardware platforms and communication protocols, making it a flexible choice for diverse IoT applications. RIOT OS offers a modular design, allowing developers to easily customise the OS to their specific needs.

Mbed OS

Mbed OS is a commercial operating system developed by Arm, focusing on security, connectivity, and ease of development. It provides a comprehensive set of APIs and tools for IoT development, making it a popular choice for commercial IoT applications. Mbed OS is well-suited for devices that require robust security and connectivity features, such as those deployed in industrial and automotive environments.

Selection criteria for an IoT OS

When selecting an IoT operating system, consider the following factors.

- **Hardware compatibility**

Processor architecture: Ensure the OS is compatible with the processor architecture of your target device (e.g., ARM, x86, RISC-V).

Memory constraints: Consider the OS's memory footprint and its ability to operate efficiently on devices with limited memory.

Peripheral support: Verify that the OS supports the specific peripherals required by your application (e.g., sensors, actuators, wireless modules).

- **Application requirements**

Real-time capabilities: If your application requires precise timing and responsiveness, choose an OS with

strong real-time capabilities, such as FreeRTOS or Zephyr.

Energy efficiency: For battery-powered devices, prioritise energy-efficient OSes like TinyOS or Contiki.

Security: If security is a critical concern, consider OSes with robust security features, such as Mbed OS or Zephyr.

Connectivity: Evaluate the OS's support for various wireless protocols (Wi-Fi, Bluetooth, Zigbee, etc) and its ability to connect to cloud platforms.

Development support and community size

Developer tools and libraries: A strong developer community and a rich ecosystem of tools and libraries can significantly accelerate development.

Documentation and tutorials: Comprehensive documentation and tutorials can help you get started quickly and efficiently.

Active community: An active community can provide support, share knowledge, and contribute to the OS's development.

Licensing and cost considerations

Open source vs commercial: Consider the licensing model of the OS. Open source OSes, like FreeRTOS and Zephyr, are often free to use, while commercial OSes may have licensing fees.

Long-term support: Ensure that the OS provider offers long-term support and updates to address security vulnerabilities and add new features.

Cost of development tools: Evaluate the cost of any additional development tools or IDEs required to work with the OS.

The challenges

Despite significant advancements, IoT operating systems still face several challenges.

- **Limited resources**

Memory constraints: IoT devices